8 | Coding principles with Dart

This chapter is a big "good practice" section as it contains some well-known suggestions from the OOP world. We'd love to also talk about design patterns, TDD, clean code and much more but these contents go beyond the scope of this book. Many people have written books and articles on these topics, we recommend you read up on these for more in-depth details.

- Design patterns are a series reusable solutions to common, well-known problems. The original concept came by a group of four people, called *Gang of four*, but nowadays new patterns come out in parallel with the evolution of languages.
  - Look for any recent book or resource that includes the widest range of patterns. They apply to any programming language; the programming language in which they're explained is not so relevant.
- TDD, abbreviation of **T**est **D**riven **D**evelopment, is a programming style strongly centered on code testing. You have to first write the tests, cover every possible case and only after this process you can start coding.
  - https://resocoder.com/flutter-clean-architecture-tdd
- DDD, abbreviation of **D**omain **D**riven **D**esign, is a programming style which focuses on code maintainability and separation of concerns. We recommend to follow Reso Coder's DDD course which gives a step-by-step explanation about DDD using Dart and Flutter.
  - https://resocoder.com/flutter-firebase-ddd-course
- Dart has a very wide, user-friendly documentation in which you can find examples for almost any topic. It's a wide growing resource that tells you how to properly write Dart code through good practices and articles.
  - https://dart.dev/guides/language/effective-dart
  - https://dart.dev/tutorials

That said, you can of course completely skip this part since it has no core Dart or Flutter concepts but we encourage you to at least know what SOLID and DI are about. These concepts are valid regardless the programming language in which they're applied.

8.1 SOLID principles

The term *SOLID* should actually be written as *S.O.L.I.D.* because it's an acronym for 5 design principles, one for each letter, which help the programmer writing maintainable and flexible code.

8.1.1 Single Responsibility Principle

Very intuitively, this principle (abbreviated with SRP) states that a class should only have a single responsibility so that it could change for one reason and no more. In other words, you should create classes dealing with a single duty so that they're easier to maintain and harder to break.

```
class Shapes {
    List<String> cache = List<>();

    // Calculations
    double squareArea(double l) { /* ... */ }
    double circleArea(double r) { /* ... */ }
    double triangleArea(double b, double h) { /* ... */ }

    // Paint to the screen
    void paintSquare(Canvas c) { /* ... */ }
    void paintCircle(Canvas c) { /* ... */ }
    void paintTriangle(Canvas c) { /* ... */ }

    // GET requests
    String wikiArticle(String figure) { /* ... */ }
    void _cacheElements(String text) { /* ... */ }
}
```

This class totally destroys the SRP as it handles internet requests, painting and calculations all in one place. You'll have to make changes very often to `Shape` because it has many duties, all in one place; maintenance for this class is not going to be pleasant. What about this?

```
// Calculations and logic
abstract class Shape {
    double area();
}
class Square extends Shape {}
class Circle extends Shape {}
class Rectangle extends Shape {}

// UI painting
class ShapePainter {}

// Networking
class ShapesOnline {}
```

There are 3 separated classes focusing on a single task to accomplish: they are easier to read, test, maintain and understand. With this approach the attention of the developer is focused on a certain area of interest (such as mathematical calculations on `Shape`) rather than on a messy collection of methods, each with different purposes.

8.1.2 Open closed principle

The open closed principle states that in a good architecture you should be able to add new behaviors without modifying the existing source code. This concept is notoriously described with the sentence "*software entities should be open for extensions but closed for modifications*". Look at this example:

```
class Rectangle {
    final double width;
    final double height;
    Rectangle(this.width, this.height);
}

class Circle {
    final double radius;
    Rectangle(this.radius);

    double get PI => 3.1415;
}
```

```
class AreaCalculator {
    double calculate(Object shape) {
        if (shape is Rectangle) {
            // Smart cast
            return r.width * r.height;
        } else {
            final c = shape as Circle;
            return c.radius * c.radius * c.PI;
        }
    }
}
```

Both `Rectangle` and `Circle` respect the SRP as they only have a single responsibility (which is representing a single geometrical shape). The problem is inside `AreaCalculator` because if we added other shapes, we would have to edit the code to add more `if` conditions.

```
class Rectangle {...}
class Circle {...}
class Triangle {...}
class Rhombus {...}
class Trapezoid {...}

class AreaCalculator {
    double calculate(Object shape) {
        if (shape is Rectangle) {
            // code for Rectangle...
        } else if (shape is Circle) {
            // code for Circle...
        } else if (shape is Triangle) {
            // code for Triangle...
        } else if (shape is Rhombus) {
            // code for Rhombus...
        } else {
            //code for Trapezoid...
        }
    }
}
```

Having added 3 new classes, the `double calculate(...)` must be changed because it requires

more `if` conditions to handle proper type casts. In general, every time that a new shape is added or removed, this method has to be maintained due to the presence of type casts. We can do better!

```
// Use it as an interface
abstract class Area {
    double computeArea();
}

// Every class calculates the area by itself
class Rectangle implements Area {}
class Circle implements Area {}
class Triangle implements Area {}
class Rhombus implements Area {}
class Trapezoid implements Area {}

class AreaCalculator {
    double calculate(Area shape) {
        return shape.computeArea();
    }
}
```

Thanks to the interface, now we have the possibility to add or remove as many classes as we want **without** changing `AreaCalculator`. For example, if we added `class Square implements Area` it would automatically be "compatible" with the `double calculate(...)` method.

> ℹ The gist of this principle is: depend on abstractions and not on implementations. Thanks to `abstract` classes you work with abstractions and not with the concrete implementations: your code doesn't rely on "predefined" entities.

8.1.3 Liskov Substitution Principle

The Liskov Substitution Principle states that subclasses should be replaceable with superclasses without altering the logical correctness of the program. In practical terms, it means that a subtype must guarantee the "usage conditions" of its supertype **plus** something more it wants to add. Look at this example:

```
class Rectangle {
    double width;
    double height;
    Rectangle(this.width, this.height);
}

class Square extends Rectangle {
    Square(double length): super(length, length);
}
```

We have a big logic problem here. A square must have 4 sides with the same length but the rectangle doesn't have this restriction. We're able to do this:

```
void main() {
    Rectangle fail = Square(3);

    fail.width = 4;
    fail.height = 8;
}
```

At this point we have a square with 2 sides of length 4 and 2 sides of length 8... which is absolutely wrong! Sides on a square must be all equal but our hierarchy is logically flawed. The LSP is broken because this architecture does **NOT** guarantee that the subclass will maintain the logic correctness of the code.

> This example also shows that inheriting from abstract classes or interfaces, rather than concrete classes, is a very good practice. Prefer composition (with interfaces) over inheritance.

To solve this problem, simply make `Rectangle` and `Square` two independent classes. Breaking LSP does not occur if you depend from interfaces: they don't provide any logic implementation as it's deferred to the actual classes.

8.1.4 Interface Segregation Principle

This principle states that a client doesn't have to be forced to implement a behavior it doesn't need. What turns out from this is: you should create small interfaces with minimal methods. Generally it's better having 8 interfaces with 1 method instead of 1 interface with 8 methods.

```
// Interfaces
abstract class Worker {
    void work();
    void sleep();
}

class Human implements Worker {
    void work() => print("I do a lot of work");
    void sleep() => print("I need 10 hours per night...");
}

class Robot implements Worker {
    void work() => print("I always work");
    void sleep() {} // ??
}
```

Robots don't need to sleep and thus the method is actually useless, but it still needs to be there otherwise the code won't compile. To solve this, let's just split `Worker` into multiple interfaces:

```
// Interfaces
abstract class Worker {
    void work();
}
abstract class Sleeper {
    void sleep();
}

class Human implements Worker, Sleeper {
    void work() => print("I do a lot of work");
    void sleep() => print("I need 10 hours per night...");
}

class Robot implements Worker {
    void work() => print("I always work");
}
```

This is definitely better because there are no useless methods and we're free to decide which behaviors should the classes implement.

8.1.5 Dependency Inversion Principle

This is very important and useful: DIP states that we should code against abstractions and not implementations. Extending an abstract class or implement an interface is good but descending from a concrete classed with no abstract methods is bad.

```
// Use this as interface
abstract class EncryptionAlgorithm {
    String encrypt(); // <-- abstraction
}

class AlgoAES implements EncryptionAlgorithm {}
class AlgoRSA implements EncryptionAlgorithm {}
class AlgoSHA implements EncryptionAlgorithm {}
```

Dependency injection (DI) is a very famous way to implement the DIP. Depending on abstractions gives the freedom to be independent from the implementation and we've already dealt with this topic. Look at this example:

```
class FileManager {
    void secureFile(EncryptionAlgorithm algo) {
        algo.encrypt();
    }
}
```

The `FileManager` class knows **nothing** about how `algo` works, it's just aware that the `encrypt()` method secures a file. This is essential for maintenance because we can call the method as we want:

```
final fm = FileManager(...);

fm.secureFile(AlgoAES());
fm.secureFile(AlgoRSA());
```

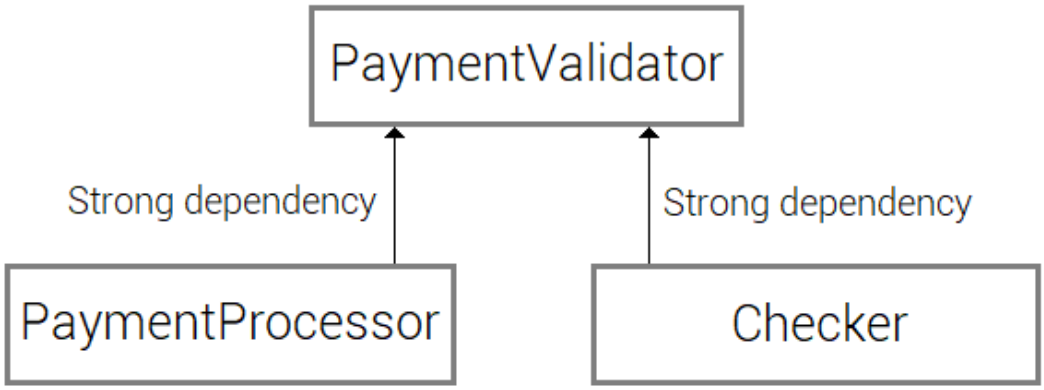
If we added another encryption algorithm, it would be automatically compatible with `secureFile` as it is a subtype of `EncryptionAlgorithm`. In this example, we're respecting the 5 SOLID principles all together.

8.2 Dependency Injection

Two classes are said to be "coupled" if at least one of them depends on the other. Class A depends on class B when you can't compile class A without the presence of class B. This can be very dangerous, let's see why.

```
class PaymentValidator {
    final Date date;
    final String cardNumber;
    const PaymentValidator(this.date, this.cardNumber);

    // Uses the MasterCard payment circuit
    void validatePayment(int amount) { ... }
}

class PaymentProcessor {
    late final _validator;
    PaymentProcessor(String cardNumber) {
      _validator = PaymentValidator(DateTime.now(), cardNumber);
    }

    Date get expiryDate => _validator.date;
    void pay(int amount) =>
        _validator.validatePayment(amount);
}

abstract class Checker {
    PaymentValidator mastercardCheck();
}
class CheckerOne extends Checker { /*... code ... */}
class CheckerTwo extends Checker { /*... code ... */}
```

Both `Checker` and `PaymentProcessor` have a strong dependency on `PaymentValidator` because it's essential in order to compile. Subclasses, of course, inherit the dependency too.



Let's say you've written this code at work. One day, your project manager tells you to ditch Mastercard and replace it with PayPal. You'll quickly get a stomach ache as soon as you realize that, from an apparently small change, the whole architecture has to be refactored.

1. Paypal just requires an email but your Mastercard implementation requires date and card number. You're forced to entirely change `PaymentValidator` but by consequence you also need to update both `PaymentProcessor` and `Checker` as they're strong dependencies.

```
class PaymentValidator {
    final String _email;
    const PaymentValidator(this._email);

    void validatePayment(int amount) { ... }
}

class PaymentProcessor {
    late final PaymentValidator _validator;

    PaymentProcessor(String email) :
        _validator = PaymentValidator(email);

    void pay(int amount) =>
        _validator.validatePayment(amount);
}
```

There's been a "cascade" effect because changes made to a single class had consequences to other classes as well.

2. The above changes break another part of the code: the abstract class `Checker` also depends on `PaymentValidator` so there's the need to fix the code.

```
abstract class Checker {
    // earlier it was called 'mastercardCheck()'
    PaymentValidator payPalCheck();
}
```

This change has consequences on `any` subclass of `Checker` which has to be updated. Dependencies on superclasses are inherited by its children and thus the coupling propagates.

3. There are no ways to solve this problem other than manually updating every single subclass of `Checker`. Your IDE will come to the rescue with a refactor tool but maintenance is a pain anyway.

As you've just seen, an apparently small and quick change on a class propagated to an entire hierarchy and other big components of our project. All of this happened because classes are strongly coupled and they depend on implementations rather than abstractions.

8.2.1 Constructor injection

Using dependency injection and abstractions rather than implementations, the above problems fade away. Dependencies passed from the outside create a *weak* coupling which is safer than a *strong* one as it relies on abstractions.

```
abstract class PaymentValidator {
    const PaymentValidator();
    void validatePayment(int amount);
}

class MasterCard implements PaymentValidator {
    // Define date, card number and the constructor
    const MasterCard();
    void validatePayment(int amount) {...}
}

class PayPal implements PaymentValidator {
    // Define an email and the constructor
```

```
    const PayPal();
    void validatePayment(int amount) {...}
}
```

The `PaymentProcessor` class is still going to have a `PaymentValidator` dependency but it's **weak** because it's just an interface. Using "constructor injection" we pass from the outside a concrete implementation, which can later be replaced with anything else.

```
class PaymentProcessor {
    final PaymentValidator _validator;
    const PaymentProcessor(this._validator);

    void pay(int amount) =>
        _validator.validatePayment(amount);
}

// And then we can freely use PayPal or MasterCard
void main() {
    final p1 = const PaymentProcessor(MasterCard());
    final p2 = const PaymentProcessor(PayPal());
}
```

In this case, we're passing an instance of a concrete class via constructor and that's **fundamental**. `PaymentProcessor` knows nothing about the implementation details of the validator object, it just knows he has to call `validatePayment(int)`. We can also use `const` constructors now!

- This code is very flexible and maintainable. If your boss told you to add support for the Visa circuit as well, you would simply have to create a new subtype of `PaymentValidator`.

  ```
  class Visa implements PaymentValidator {
      const Visa();
      void validatePayment(int amount) {...}
  }
  ```

  No changes are required to the existing code and you at the same time you're still embracing S.O.L.I.D. principles. The architecture is robust!

- `PaymentProcessor` now doesn't care anymore about Mastercard, Paypal or whatever because they're given from the outside. Internally he weakly depends on an abstraction which just gives an abstraction: the implementation is passed via constructor.

- You have a series of classes, one per payment method, that are super easy to test. You

could make a "mock" class for unit tests just like a regular validator type:

```
class TestValidator implements PaymentValidator {
    const TestValidator();
    void validatePayment(int amount) {...}
}
```

Last thing we need to refactor is the `Checker` class as it has to return an abstraction rather than an implementation.

```
abstract class Checker {
    PaymentValidator paymentCheck();
}

class CheckerOne extends Checker {...}
class CheckerTwo extends Checker {...}
```

Since `PaymentValidator` is `abstract`, any class along the hierarchy inherits a *weak* dependency which is safe.

8.2.2 Method injection

Constructor injection is used when you class really needs an external dependency to work. When you have an "optional" dependency not strictly required from your class, you can use method injection.

```
abstract class CheckProcessor {
    const CheckProcessor();
    bool isActive();
}

class MastercardCheck implements CheckProcessor {
    final MasterCardApi _api;
    const MastercardCheck(this._api);

    bool isActive() async => await _api.isOnline();
}

class PaypalCheck implements CheckProcessor {
    final PaypalApi _api;
```

```
    const PaypalCheck(this._api);

    bool isActive() async => await _api.available();
}
```

These classes connect to the internet, perform some GET requests and return `true` or `false` whether the service provider is online or not. Let's say this feature is not essential in our architecture but it's nice to have it. It might be used but it's not certain.

```
class PaymentProcessor {
    final PaymentValidator _validator;
    const PaymentProcessor(this._validator);

    void pay(int amount) => ...

    bool isProcessorActive(CheckProcessor check) =>
        return check.isActive();
}
```

In this way, if we wanted to check the availability of the service we could do this:

```
void main() {
    final api = MasterCardApi(...);
    final processor = MasterCard(api);
    final checker = MastercardCheck();

    final payment = PaymentProcessor(processor);
    final isOnline = payment.isProcessorActive(checker);
}
```

Note the difference: while the processor is fundamental, and thus it's passed via constructor, the connection checker made with `isProcessorActive` is not always required. So in general:

- constructor injection is for essential dependencies that your class is **always** going to use;
- method injection is for optional dependencies that you class **might** use.

Actually both type of injection use the same concept, which is depending on abstractions and passing implementations from the outside, but they differ in order of "importance". Dependencies passed via constructor are fundamental while the ones passed via method are just useful but not essential.